Cairo University

Faculty of Computers and Artificial Intelligence

CS251

Introduction to Software Engineering

FinTrix

Software Design Specifications

Version 1.1

ID	Name	Email	Phone number
20242227	Omar Rady Tammam	20242227@stud.fci-cu.edu.eg	01155537979
20240084	Asmaa Sameh Mohamed	20240084@stud.fci-cu.edu.eg	
20240067	Adham Tamer Abdelmaougoud	20240067@stud.fci-cu.edu.eg	
20240596	Mostafa Mohamed Ahmed	20240596@stud.fci-cu.edu.eg	

May of 2026

CS251: <Team Name>
Project: <Project Name>

Software Design Specification

Contents

Team 3

Document Purpose and Audience 3

Purpose of the Document 3

Target Audience 3

System Models 4

I. Architecture Diagram 4

II. Class Diagram(s) 5

III. Class Descriptions 6

IV. Sequence diagrams 9

Class - Sequence Usage Table 17

V. State Diagram 19

VI. SOLID Principles 19

VII. Design Patterns 19

Tools 19

Ownership Report 19

CS251: <Team Name>
Project: <Project Name>

Software Design Specification

Team

ID	Name	Email	Phone number
20242227	Omar Rady Tammam	20242227@stud.fci-cu.edu.eg	01155537979
20240084	Asmaa Sameh Mohamed	20240084@stud.fci-cu.edu.eg	
20240067	Adham Tamer Abdelmaougoud	20240067@stud.fci-cu.edu.eg	
20240596	Mostafa Mohamed Ahmed	20240596@stud.fci-cu.edu.eg	

Document Purpose and Audience

Purpose of the Document

This document provides a detailed description of the system design for the Personal Finance Management Application.
It includes sequence diagrams, system workflows, and the interaction between different components such as UI, services, controllers, and database.

The goal of this document is to:

- Explain how the system works internally
- Show how different features are implemented (e.g., login, transactions, goals, reports)
- Ensure that all system components and methods are clearly defined and connected
- Serve as a reference for development and future maintenance

Target Audience

This document is intended for:

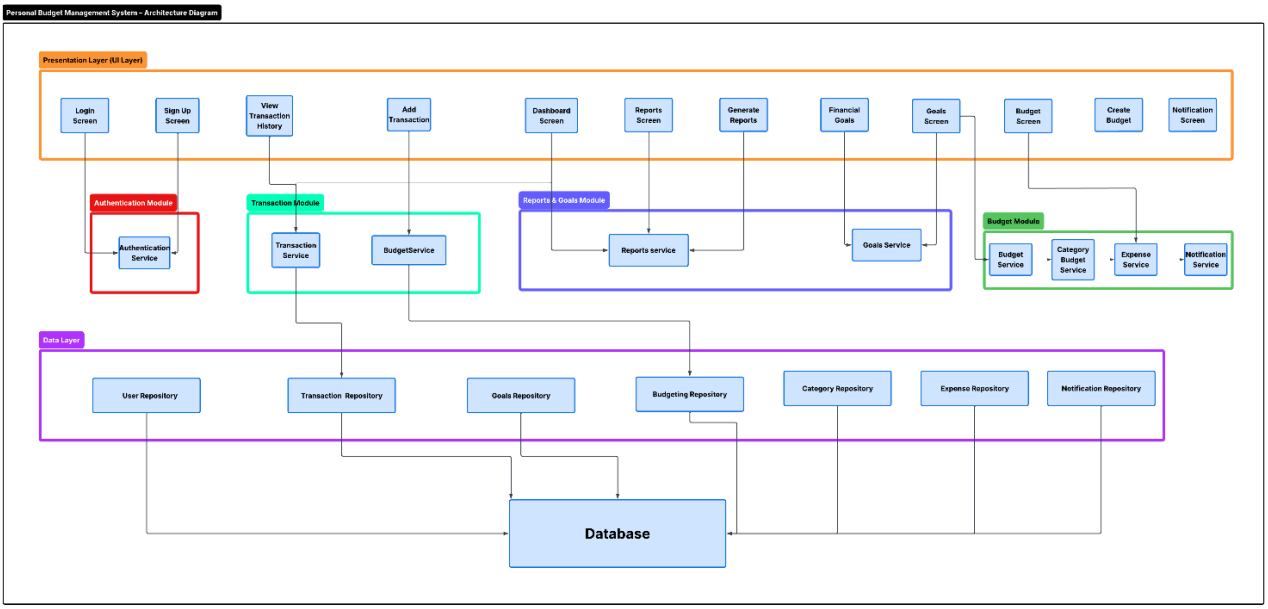
- **Developers**: to understand system structure, logic, and implementation details
- **Project Managers**: to track system progress and ensure requirements are met
- **System Designers / Architects**: review design decisions and system structure

Software Design Specification

- **Testers (QA Engineers)**: to understand workflows and test different scenarios
- **Instructors / Evaluators**: to assess the correctness and completeness of the system design

System Models

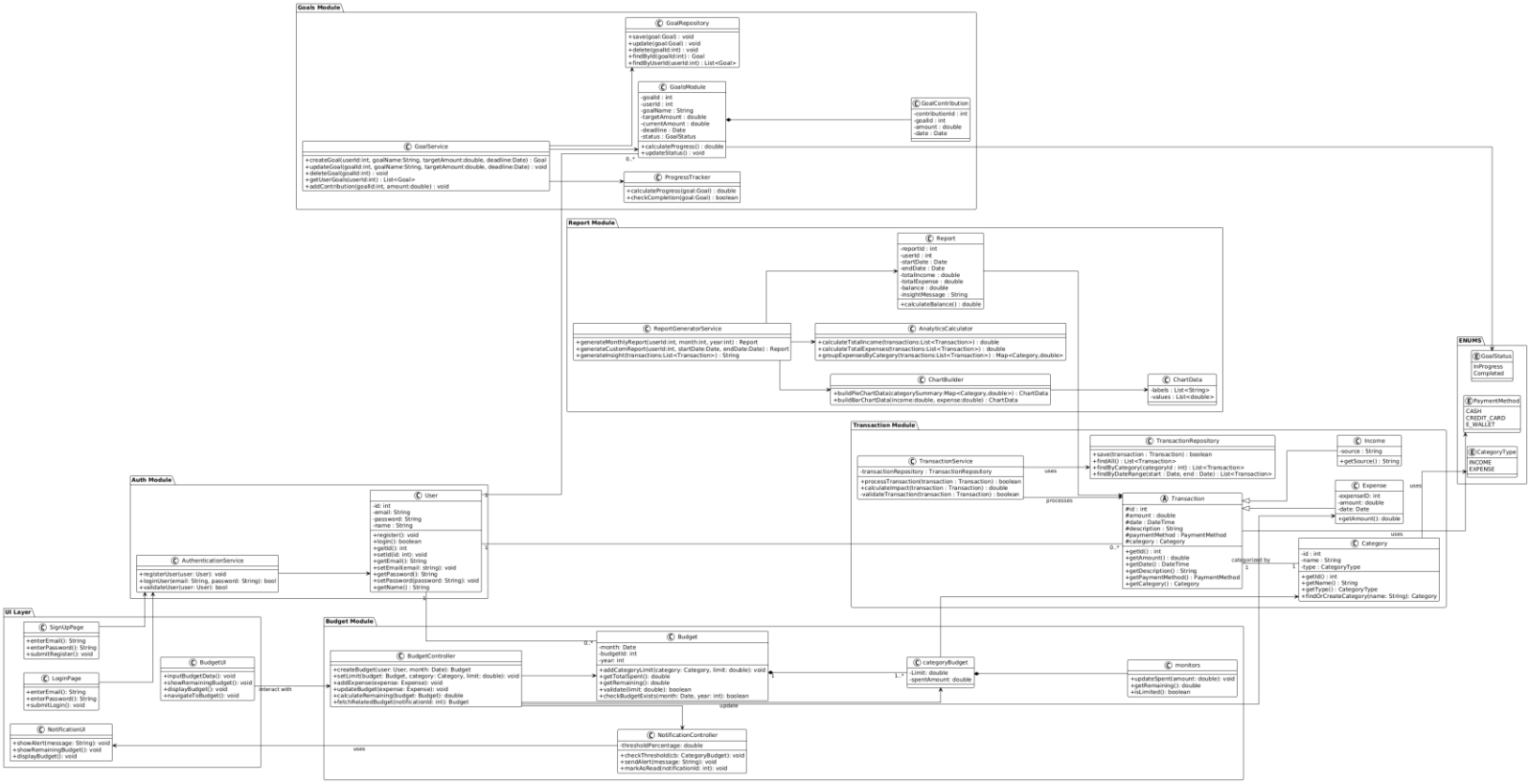
I. Architecture Diagram



CS251: <Team Name>
Project: <Project Name>

Software Design Specification

II. Class Diagram(s)



Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1	**LoginPage**	Handles user login interface interactions. Responsible for receiving user email and password input and submitting login requests to the authentication service.
2	**SignUpPage**	Handles user registration interface. Responsible for collecting user credentials and submitting registration requests.
3	**BudgetUI**	Handles all budget-related user interface interactions. Responsible for receiving user input for budget data, displaying the current budget, showing remaining budget, and navigating to the budget screen.
4	**NotificationUI**	Handles the notification-related user interface. Responsible for displaying alerts and budget notifications to the user.
5	**User**	Represents a system user. Responsible for storing user data such as id, email, password, and name, and performing operations like login and registration.
6	**AuthenticationService**	Handles authentication logic. Responsible for registering users, validating credentials, and managing login operations.
7	**Transaction**	Abstract class representing a financial transaction. Responsible for storing common transaction details such as id, amount, date, description, payment method, and category.
8	**Income**	Represents an income transaction. Responsible for storing income source and extending the base transaction functionality.
9	**Expense**	Represents an expense transaction. Responsible for storing expense details and providing access to expense amount.
10	**Category**	Represents a spending category. Responsible for storing category information and finding or creating a category by name.
11	**TransactionService**	Handles business logic related to transactions. Responsible for processing transactions, validating them, and calculating their financial impact.
12	**TransactionRepository**	Handles data persistence for transactions. Responsible for storing, retrieving, and filtering transaction data.

Software Design Specification

13	**Budget**	Represents a monthly budget entity. Responsible for storing budget details, adding category limits, calculating total spent, and getting remaining balance.
14	**CategoryBudget**	Represents the budget allocated for a specific category. Responsible for tracking spending limits and spent amount.
15	**monitors**	Responsible for tracking budget consumption. Handles updating spent amounts, calculating remaining balance, and checking limits.
16	**BudgetController**	Acts as the main controller for budget operations. Responsible for creating budgets, setting category limits, updating expenses, and calculating remaining balance.
17	**NotificationController**	Manages notification logic. Responsible for monitoring budgets, checking thresholds, sending alerts, and marking notifications as read.
18	**GoalsModule**	Represents a financial goal. Responsible for storing goal data, tracking progress, and updating goal status.
19	**GoalContribution**	Represents a contribution made toward a goal. Responsible for storing contribution amount, date, and associated goal.
20	**GoalService**	Handles goal-related operations such as creating, updating, deleting goals, and adding contributions.
21	**ProgressTracker**	Responsible for calculating goal progress and checking completion status.
22	**GoalRepository**	Handles persistence of goals. Responsible for saving, updating, deleting, and retrieving goal data.
23	**Report**	Represents a financial report. Responsible for storing report data and calculating balance.
24	**ReportGeneratorService**	Handles report generation logic, including creating reports and generating insights.
25	**ChartBuilder**	Responsible for preparing chart data for visualization such as pie and bar charts.
26	**AnalyticsCalculator**	Performs financial calculations such as total income, expenses, and category grouping.
27	**ChartData**	Represents chart data structure containing labels and values.

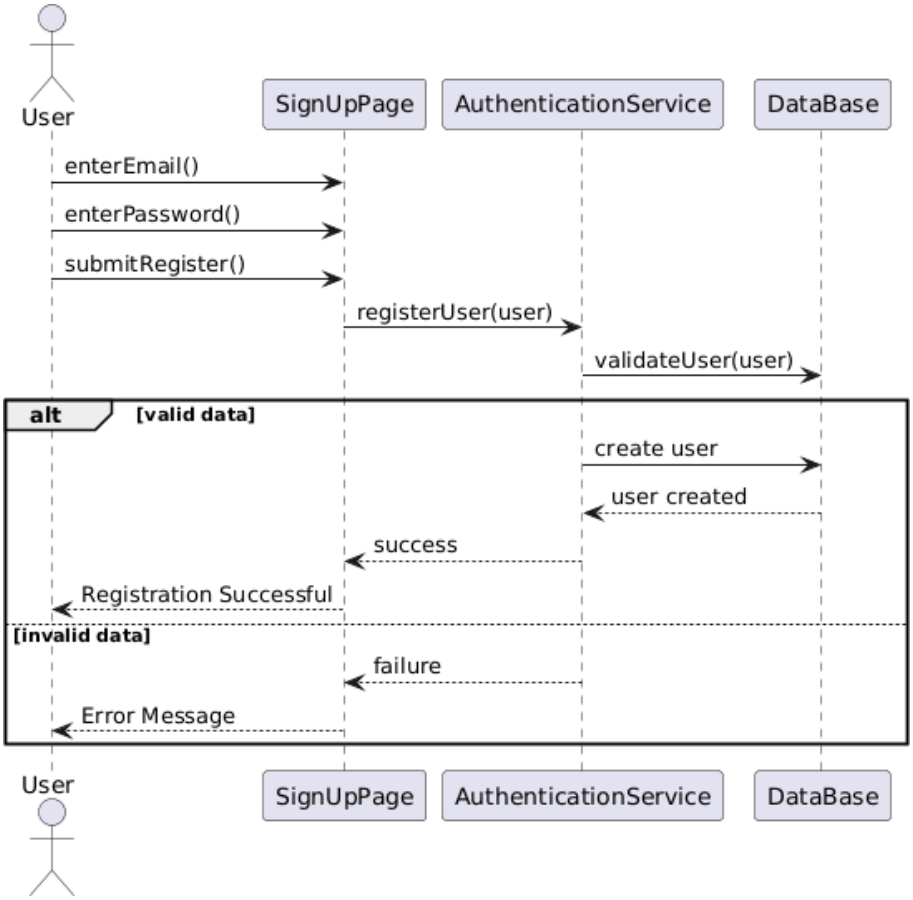
Software Design Specification

28	**CategoryType**	Enum defining category type (Income or Expense).
29	**PaymentMethod**	Enum defining payment methods (Cash, Credit Card, E-Wallet).
30	**GoalStatus**	Enum defining goal status (In Progress or Completed).

Software Design Specification

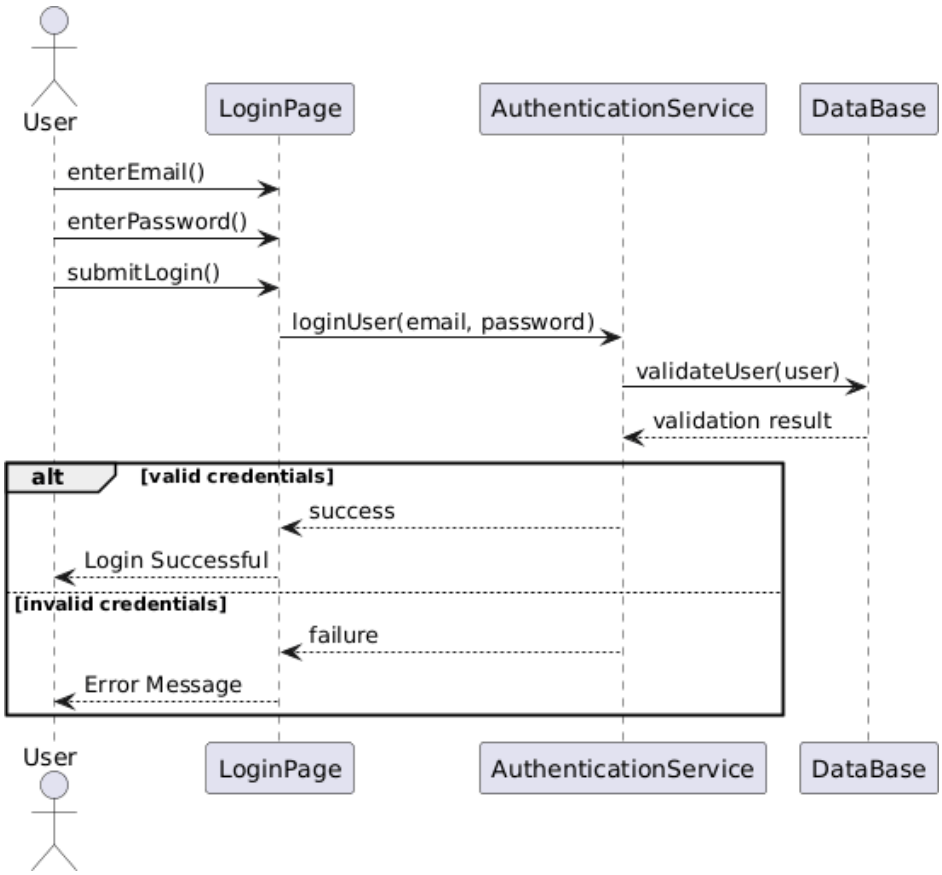
IV. Sequence diagrams

Sequence Diagram : User Sign-Up (US #1)

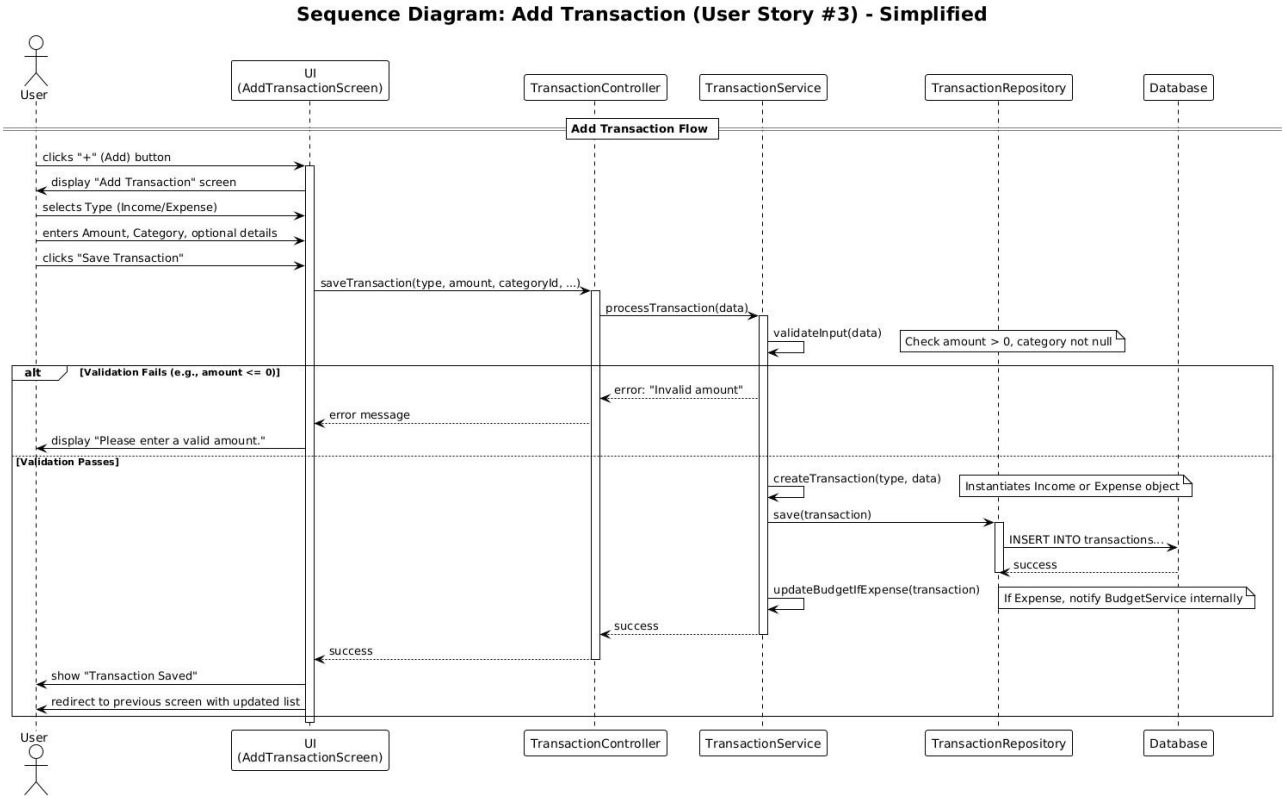


Software Design Specification

Sequence Diagram : User Log-In (US #2)

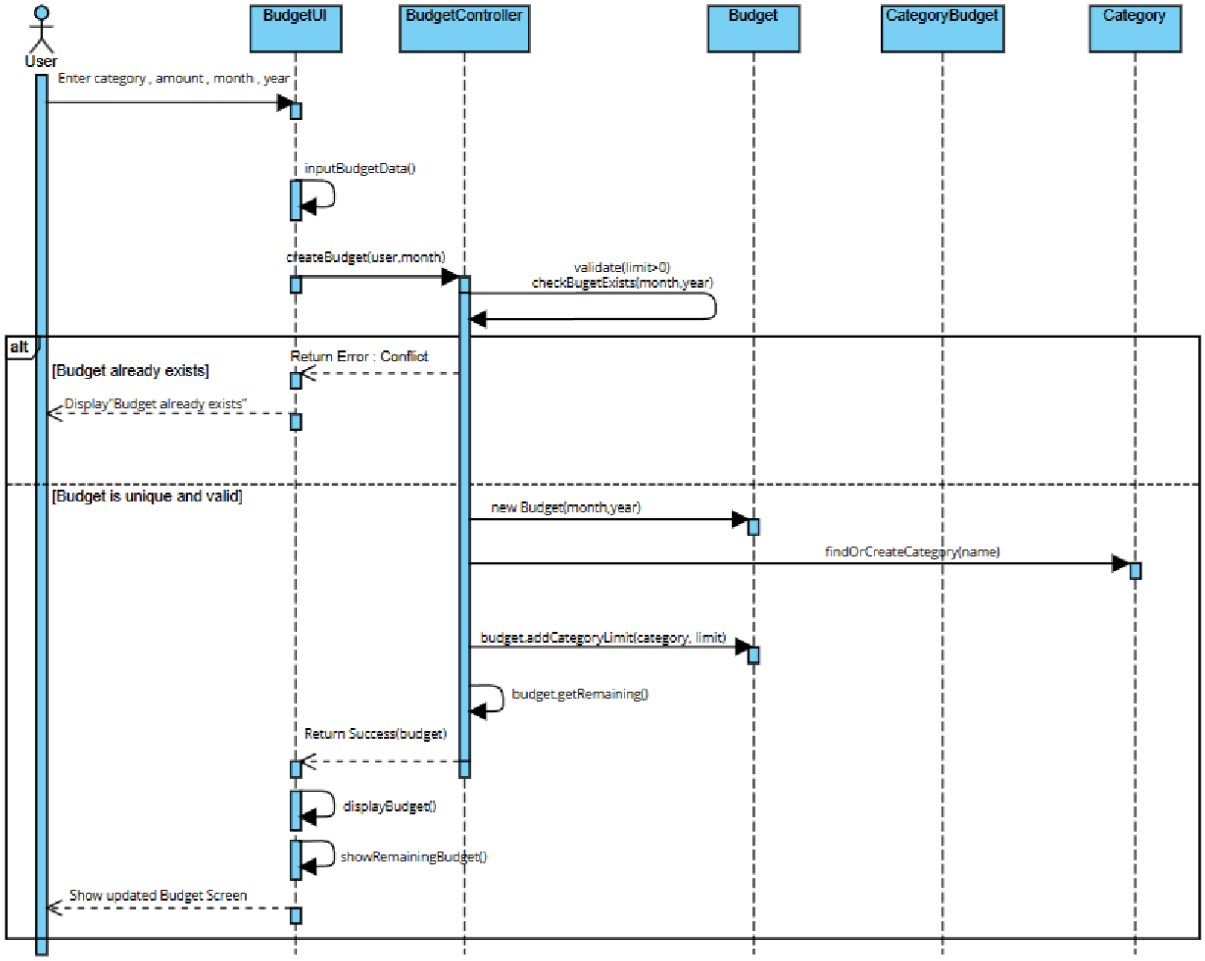


Software Design Specification

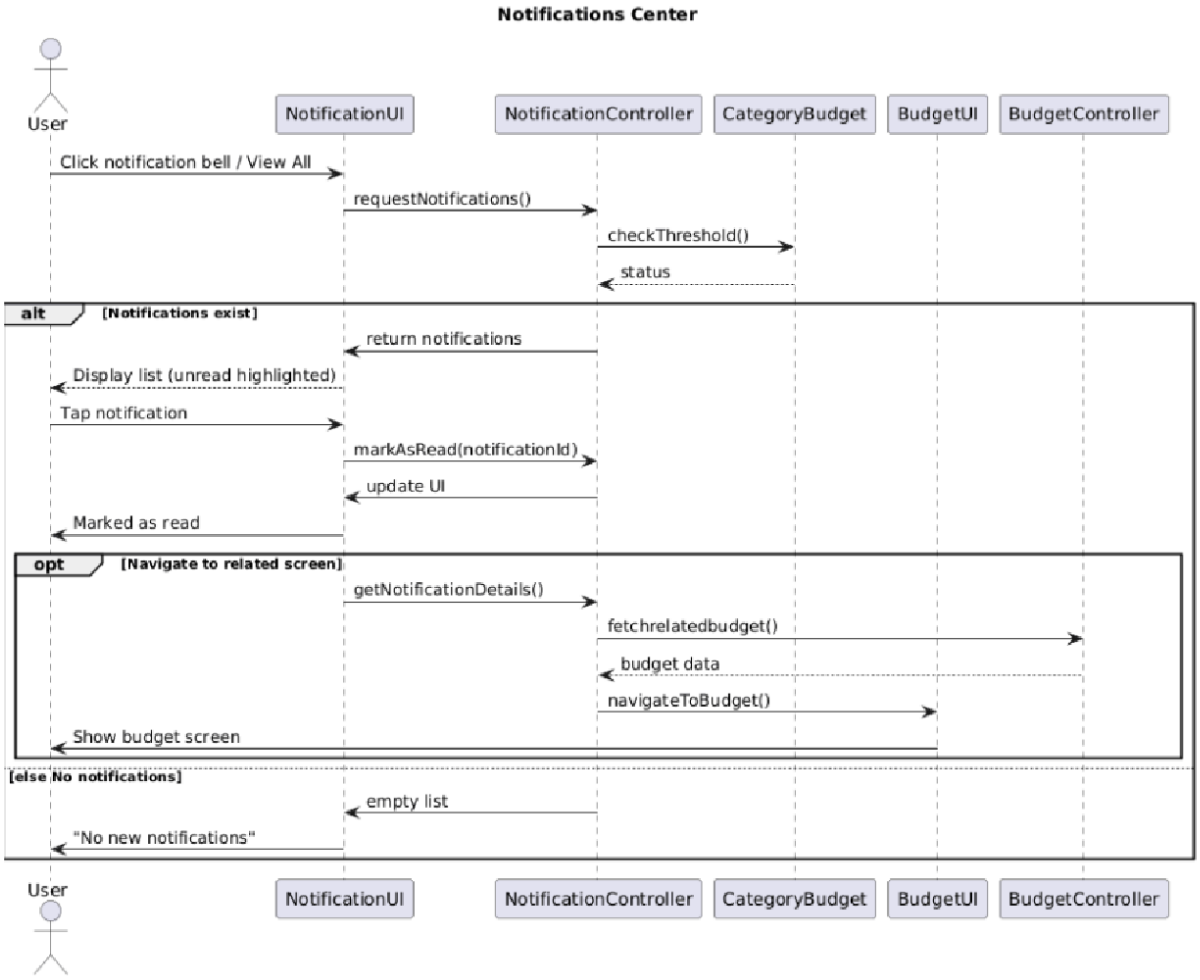


Software Design Specification

Sequence Diagram : Create/Edit Budget (US #4)

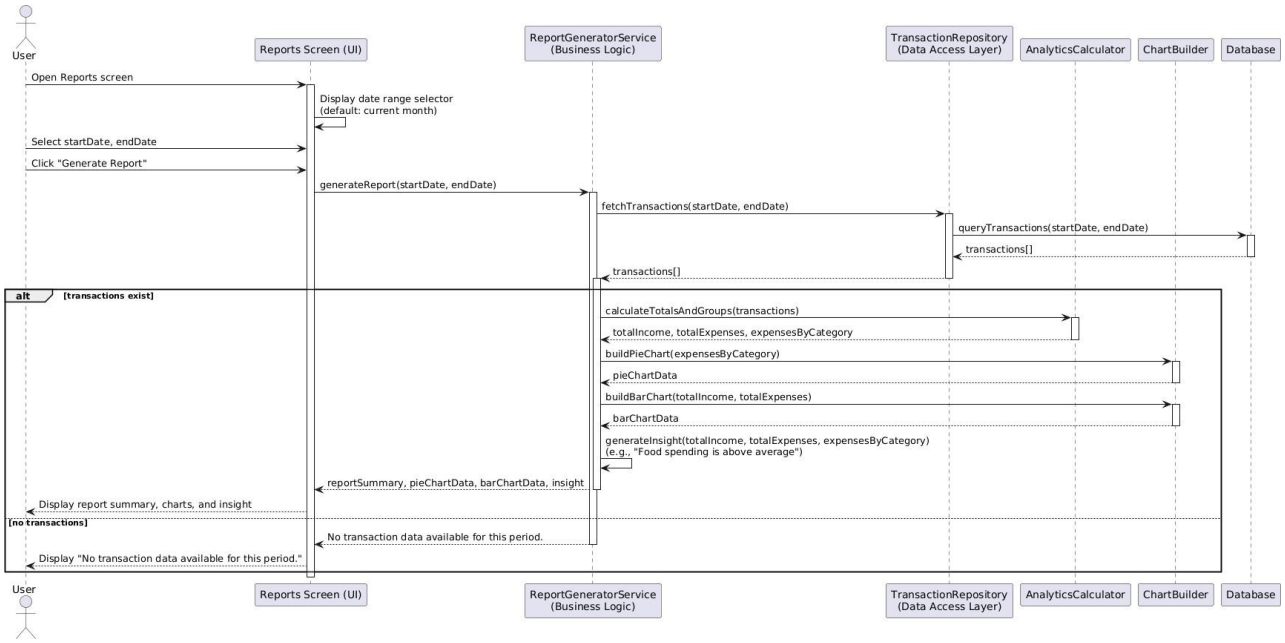


Software Design Specification



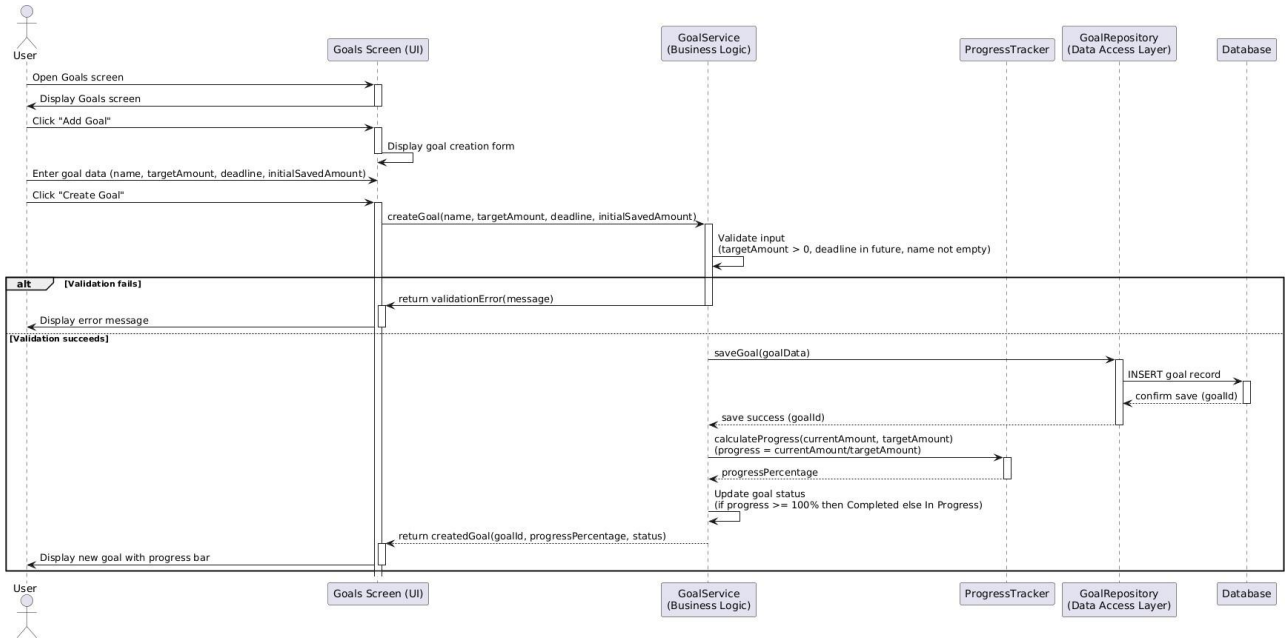
Software Design Specification

Sequence Diagram : View Reports and Analytics (US #7)



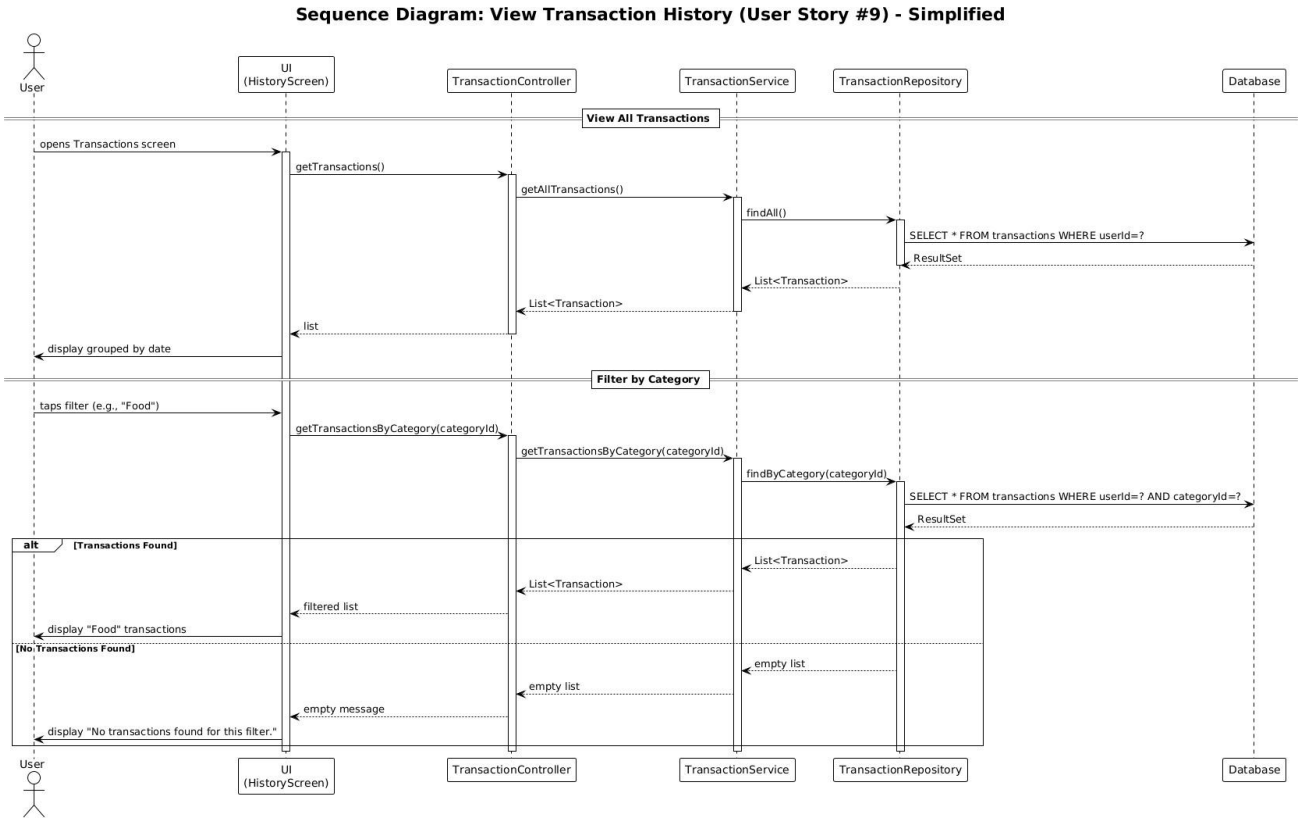
Software Design Specification

Sequence Diagram : Set and Track Financial Goals (US #6)



Sequence Diagram : View Reports and Analytics (US #7)

Software Design Specification



Software Design Specification

Class - Sequence Usage Table

Sequence Diagram	Class	Methods Used
Login	LoginPage	enterEmail(), enterPassword(), submitLogin()
	AuthenticationService	loginUser(email, password)
	UserEntity	validateUser(user)
Sign Up	SignUpPage	enterEmail(), enterPassword(), submitRegister()
	AuthenticationService	registerUser(user)
	UserEntity	validateUser(user), createUser()
Add Transaction	AddTransactionScreen (UI)	displayAddTransaction(), collectInput(), showSuccess(), showError()
	TransactionController	saveTransaction(type, amount, categoryId)
	TransactionService	processTransaction(data), validateInput(data), createTransaction(type, data), updateBudgetIfExpense(transaction)
	TransactionRepository	save(transaction)
	Database	insertTransaction()
View Transactions	HistoryScreen (UI)	getTransactions(), displayTransactions(), displayFilteredTransactions(), showEmptyMessage()
	TransactionController	getTransactions(), getTransactionsByCategory(categoryId)
	TransactionService	getAllTransactions(), getTransactionsByCategory(categoryId)
	TransactionRepository	findAll(), findByCategory(categoryId)
	Database	selectTransactions(), selectTransactionsByCategory()

Software Design Specification

Goals Management	Goals Screen (UI)	displayGoals(), displayGoalForm(), showError(), displayGoal()
	GoalService	createGoal(name, targetAmount, deadline, initialSavedAmount), validateInput(), updateGoalStatus()
	GoalRepository	saveGoal(goalData)
	ProgressTracker	calculateProgress(currentAmount, targetAmount)
	Database	insertGoal()
Reports	ReportsScreen (UI)	displayReport(), displayNoData()
	ReportGeneratorService	generateReport(startDate, endDate), generateInsight()
	TransactionRepository	fetchTransactions(startDate, endDate)
	AnalyticsCalculator	calculateTotalsAndGroups(transactions)
	ChartBuilder	buildPieChart(), buildBarChart()
	Database	queryTransactions()
Notifications Center	NotificationUI	showAlert(message)
	NotificationController	checkThreshold(cb), markAsRead(notificationId), sendAlert(message)
	CategoryBudget	checkThreshold(cb)
	BudgetUI	navigateToBudget()
	BudgetController	fetchRelatedBudget(notificationId)

Software Design Specification

Budget Creation	BudgetUI	inputBudgetData(), displayBudget(), showRemainingBudget()
	BudgetController	createBudget(user, month)
	Budget	validate(limit), checkBudgetExists(month, year), addCategoryLimit(category, limit), getRemaining()
	Category	findOrCreateCategory(name)

Software Design Specification

V. State Diagram

VI. SOLID Principles

1. Single Responsibility Principle (SRP)

The Single Responsibility Principle states that a class should have only one responsibility and one reason to change.
This principle is clearly applied in the separation between TransactionService and TransactionRepository.

The TransactionService class is responsible only for handling business logic such as transaction validation, processing, and financial calculations. On the other hand, the TransactionRepository class is responsible only for data persistence operations such as saving transactions and retrieving them from the database.

This separation improves maintainability and reduces coupling between system components. For example, if the database implementation changes, only TransactionRepository needs modification. Similarly, if transaction validation rules change, only TransactionService is affected.

The SRP principle is also applied in several other modules of the system. AuthenticationService handles authentication logic only, ReportGeneratorService focuses only on generating reports and analytics, while NotificationController manages notification operations independently from the budgeting logic.

2. Open/Closed Principle (OCP)

The Open/Closed Principle states that software entities should be open for extension but closed for modification.

This principle is applied in the Transaction abstract class and its subclasses Income and Expense. The Transaction class defines the common attributes and behaviors shared between all transaction types such as id, amount, date, description, paymentMethod, and category.

Instead of modifying the base Transaction class whenever a new transaction type is needed, the system can simply extend it by creating a new subclass. The existing code remains unchanged while new functionality is added through inheritance and polymorphism.

This design makes the system scalable and flexible. Future transaction types such as TransferTransaction or InvestmentTransaction can be introduced without modifying existing services or repositories.

3. Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that subclasses must be replaceable with their parent class without affecting the correctness of the program.

This principle is applied in the relationship between the abstract Transaction class and its subclasses Income and Expense. Both subclasses inherit the common structure and behavior of Transaction and can be used anywhere a Transaction object is expected.

For example, TransactionService and TransactionRepository work with objects of type Transaction without needing to know whether the actual object is an Income or an Expense.

This approach improves abstraction and flexibility in the system. It allows the application to process different transaction types uniformly while preserving correct behavior and reducing dependency on concrete implementations.

4. Interface Segregation Principle (ISP)

The Interface Segregation Principle states that classes should not be forced to depend on methods they do not use.

This principle appears in the system through the separation of responsibilities between controllers and services. For example, AuthenticationService contains only authentication-related methods such as registerUser(), loginUser(), and validateUser(), while budgeting operations are handled separately inside BudgetController.

Similarly, reporting operations are isolated inside ReportGeneratorService, and goal-tracking logic is managed by GoalService and ProgressTracker.

By separating functionalities into focused classes, each component depends only on the operations it actually requires. This keeps the system modular, reduces unnecessary dependencies, and improves readability and maintainability.

5. Dependency Inversion Principle (DIP)

The Dependency Inversion Principle states that high-level modules should not depend directly on low-level modules. Both should depend on abstractions.

This principle is applied in the interaction between service classes and repository classes. For example, TransactionService depends on the abstraction of transaction storage operations rather than handling database operations directly.

Instead of embedding persistence logic inside the business layer, the system delegates storage responsibilities to TransactionRepository. The same concept is applied in modules such as goals and reports where services communicate with repositories and helper classes.

This design improves flexibility and testability. If the storage mechanism changes from SQLite to PostgreSQL or an external API, the business logic layer remains unaffected because dependencies are separated properly.

VII. Design Patterns

1) Strategy Pattern

The Strategy Pattern defines a family of behaviors and makes them interchangeable depending on the object type. In the FinTrix system, this pattern is applied in the Transaction abstract class and its subclasses Income and Expense. The system defines a general transaction structure while each subclass handles a different transaction behavior. Income is responsible for income-related operations, while Expense handles expense-related operations. Using this pattern removes the need for large conditional statements and makes the system easier to extend in the future by adding new transaction types without modifying the existing logic.

2) Factory Method Pattern

The Factory Method Pattern provides a centralized way to create objects without exposing the exact concrete class to the client. This pattern is applied in TransactionService, where transactions are created depending on the provided type. Instead of creating Income and Expense objects directly in multiple places, the creation process is handled inside one dedicated method such as createTransaction(type, data). This approach reduces coupling between classes and simplifies future expansion of the transaction system.

Software Design Specification

3) Repository Pattern

The Repository Pattern separates database operations from business logic. In the FinTrix system, this pattern is applied in TransactionRepository and GoalRepository. These classes are responsible for handling database operations such as saving, updating, and retrieving data. Instead of placing database queries inside services or controllers, repositories isolate data access logic in separate classes. This improves code organization, maintainability, and testing.

4) Observer Pattern (Conceptual)

The Observer Pattern defines a relationship where changes in one object automatically trigger updates in dependent objects. In FinTrix, this pattern is conceptually applied between CategoryBudget, NotificationController, and NotificationUI. When the budget reaches a specific threshold, the notification controller automatically sends an alert message, and the UI displays the notification to the user. This design keeps the budget logic independent from the notification system while allowing automatic reactions to budget changes.

5) MVC Pattern (Model–View–Controller)

The MVC Pattern separates the application into three main components: Model, View, and Controller. In the FinTrix system, models include classes such as User, Transaction, Budget, GoalsModule, and Report. The view layer includes UI classes such as LoginPage, SignUpPage, BudgetUI, and NotificationUI. Controllers and services such as AuthenticationService, BudgetController, GoalService, and ReportGeneratorService handle the business logic and communication between the UI and the data layer. This architecture improves scalability, readability, and maintainability of the system.

6) Service Layer Pattern

The Service Layer Pattern moves business logic into dedicated service classes instead of placing it directly inside models or UI classes. In FinTrix, this pattern is applied in classes such as AuthenticationService, TransactionService, GoalService, and ReportGeneratorService. These services handle the core business operations of the system, making the code cleaner, easier to maintain, and more reusable across different parts of the application.

7) Composition Pattern

The Composition Pattern represents a strong “has-a” relationship between objects. In FinTrix, this pattern appears in the relationship between Budget and categoryBudget, as well as between GoalsModule and GoalContribution. A budget contains multiple category budgets, and a goal

contains multiple contributions. This relationship ensures that child objects belong directly to their parent object and are managed as part of its lifecycle.

8) Builder Pattern (Conceptual)

The Builder Pattern is used to construct complex objects step by step. In the FinTrix system, this pattern is conceptually applied in ChartBuilder. The class is responsible for generating chart data for pie charts and bar charts used in reports and analytics. Instead of constructing complex chart objects directly, the builder organizes the creation process in a cleaner and more flexible way.

9) Facade Pattern

The Facade Pattern provides a simplified interface to a complex subsystem. In FinTrix, this pattern is applied in ReportGeneratorService. The service internally communicates with several classes such as AnalyticsCalculator, ChartBuilder, and Report, while exposing simple methods like generateMonthlyReport() and generateCustomReport(). This makes report generation easier to use and reduces complexity for other parts of the system.

10) Singleton Pattern (Conceptual)

The Singleton Pattern ensures that only one instance of a class exists during the application runtime. In the FinTrix system, this pattern is conceptually applied in classes such as AuthenticationService and NotificationController. These classes act as centralized services shared across the application. Using a singleton-like approach helps maintain consistent behavior and reduces unnecessary object creation.

CS251: <Team Name>
Project: <Project Name>

Software Design Specification

Tools

PlanetUML , ArgoUML , Visual-Paradigm , lucid chart

Ownership Report

Owner	Item
Omar Rady	Document Purpose and Audience
	Architecture Diagram (UI Layer & Authentication Flow)
	UI Layer Classes in Class Diagram
	Authentication Module Classes
	Sequence Diagrams: User Login & User Sign-Up
	State Diagram (User Authentication Lifecycle)
	Authentication & UI Frontend Design
	Project Structure Organization
	GitHub Repository Setup & Frontend Integration
Adham Tamer	Transaction Module Class Diagram
	Sequence Diagrams: Add Transaction & View Transaction History
	TransactionService and TransactionRepository Design
	Transaction Processing Logic
	CSV Export Feature
Mostafa Mohamed	Budget Module Class Diagram
	Sequence Diagrams: Create/Edit Budget & Notifications
	BudgetController and NotificationController Design
	Budget Monitoring Logic

CS251: <Team Name>
Project: <Project Name>

Software Design Specification

	Notification System Integration
Asmaa Sameh	Goals Module Class Diagram
	Reports Module Class Diagram
	Sequence Diagram: Financial Goals
	Sequence Diagram: Reports & Analytics
	Goal Tracking & Progress Logic
	Reports Analytics & Chart Generation
	PDF Report Export Feature
All Team Members	System Testing & Debugging
	Documentation Review
	Presentation Preparation
	SOLID Principles Documentation
	Design Patterns Documentation
	Final System Integration
	GitHub Commits & Version Control